

Tema 5. Sistemas de Gestión de Bases de Datos.

5.1 Introducción: Justificación del uso de las Bases de Datos

5.1.1 Evolución en el tratamiento de la información.

En casi todos los ámbitos del mundo actual, es fundamental el procesamiento de datos para obtener información que permita tomar decisiones.

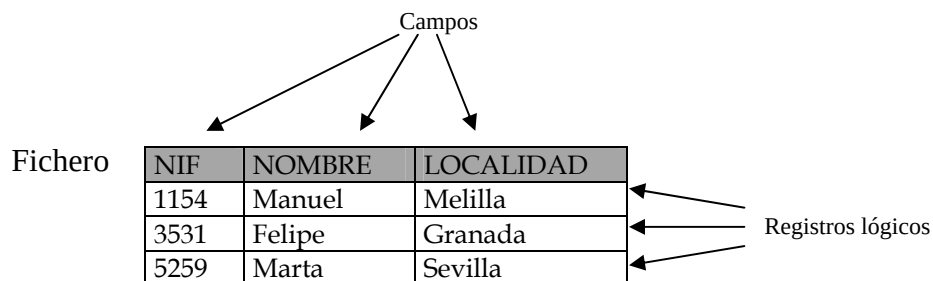
El mundo empresarial y administrativo cada vez utilizan más datos, lo cual dificulta su procesamiento. A partir de los años sesenta varias empresas empiezan a almacenar sus datos en soportes magnéticos para ser tratados directamente por computadoras. Desde entonces el auge de los equipos informáticos y soportes de información ha ido aumentando a pasos agigantados, y de igual forma han ido disminuyendo su coste.

En la evolución del tratamiento automático de la información podemos ver varias etapas: Las primeras aplicaciones integraban los datos en los programas en forma de constantes y sólo se analizaban pequeñas parcelas del mundo científico. Posteriormente aparecen los ficheros, que representan pequeñas parcelas del mundo real. En un principio estos ficheros sólo podían ser accedidos de forma secuencial y posteriormente de forma aleatoria. Las aplicaciones estaban constituidas por programas independientes y cada uno de ellos procesaba sus propios ficheros. Por ello, algunos datos tenían que ser repetidos en distintos archivos, pudiendo producirse incoherencias entre los mismos. A finales de los años sesenta, para evitar estos problemas, surgen las bases de datos, que pretenden modelar grandes parcelas del mundo real mediante la interrelación de varios archivos, evitando las redundancias y posibles incoherencias.

5.1.2. Características de los ficheros convencionales

El ordenador no sólo trabaja con datos individuales y aislados, sino con conjuntos de datos lógicamente relacionados. Se denomina **registro lógico** a un conjunto de datos referentes a una misma entidad y que constituyen una unidad para el proceso. A cada uno de los datos que forman un registro lógico se le denomina **campo** o **atributo**.

Llamamos **archivo** a un conjunto de registros (lógicos) del mismo tipo, que posee una organización coherente y que se ubica en dispositivos de almacenamiento secundario.



Se llama **registro físico** o **bloque** al conjunto de datos transferidos en una sola operación de entrada/salida. Se conoce como **factor de bloqueo** al número de registros lógicos que hay en cada registro físico o bloque.

Los registros pueden ser de **longitud fija** o de **longitud variable**. La utilización de registros de longitud fija provoca pérdida de espacio, pero son más fáciles de manejar. Los registros de



longitud variable provocan menos pérdida de espacio pero su utilización será más difícil al tener que utilizar marcas de principio y final de registro, o campos extra con la longitud del registro.

Asociado a los archivos de un soporte suele existir un *directorio de archivos*, se trata de un archivo, donde para cada archivo del soporte, existe un registro de directorio. Este registro contiene información acerca de la posición y el formato de los registros que forman el archivo. Según el tipo de organización el contenido de este registro será diferente. Normalmente el contenido de cada registro directorio suele ser el nombre del archivo, el propietario, el punto inicial, el punto final, la cantidad de espacio permitida y la utilizada actualmente. El propietario de este archivo directorio es el Sistema Operativo.

Se denomina *apertura de un fichero* al proceso de leer el registro directorio asociado al fichero y retener esta información para su consulta posterior. El proceso contrario, *cerrar el archivo*, consiste en la actualización del directorio de archivo correspondiente al fichero, en el caso que haya sufrido alguna modificación.

Tipos de acceso: Se denomina acceso al proceso físico de localizar la información contenida en el fichero. Podrá ser secuencial o directo:

- ***Acceso secuencial:*** Se accede a los registros siguiendo su orden físico de aparición en el soporte de entrada.
- ***Acceso al azar o directo:*** Se accede a los registros atendiendo a un orden aleatorio de solicitud de los mismos. Este orden es externo al programa y no predecible.

El tipo de acceso dependerá directamente del *tipo de soporte* que se utilice:

- ***Soporte de acceso secuencial:*** Para acceder a un registro físico estamos obligados a acceder a todos los registros físicos que le preceden. Por ejemplo: una cinta magnética.
- ***Soporte de acceso directo:*** Son los que permiten el acceso a cualquier registro físico una vez conocida su dirección. Por ejemplo: un disco magnético. Los soportes de acceso directo también admiten accesos secuenciales.

Tipo de proceso: Dependerá de la aplicación que procese el archivo. Podrá ser de dos tipos:

- ***En lote o batch:*** En general son los que no requieren la intervención del usuario. Un caso típico son las actualizaciones de un fichero maestro a partir de uno de movimientos. En estos procesos todos los movimientos (altas, bajas, y modificaciones) sobre un fichero denominado maestro son acumulados durante cierto tiempo en un fichero de movimientos, que posteriormente es procesado actualizando al fichero maestro. Los procesos en lote son eficaces siempre que no sea necesario que el fichero maestro esté continuamente actualizado.
- ***En línea:*** En los procesos en línea los movimientos o alteraciones del fichero son tratadas en el momento en que se producen. Estos procesos se caracterizan por permitir el diálogo con el usuario, se dice que son interactivos o conversacionales.

5.1.2.1 Clasificación de los ficheros según su uso.

Los archivos se clasifican según la utilización que se hace de ellos en tres grupos:

- **Archivos permanentes.** Contienen información que varía poco a lo largo del tiempo. Su contenido es esencial para la aplicación que los utiliza. Deben protegerse ante alteraciones accidentales y realizar copias de seguridad de forma periódica para garantizar la reconstrucción en caso de pérdida total o parcial. Pueden ser de tres clases:
 - **Archivos de constantes.** Su información permanece prácticamente inamovible, utilizándose principalmente como archivos de consulta. Un archivo de este tipo puede ser el archivo de datos personales de empleados, que suele sufrir escasas modificaciones.

- **Archivos de situación o maestros.** Reflejan el estado o situación de una empresa, entidad o algún aspecto de ella en un determinado momento. Estos archivos se actualizan periódicamente para adaptarlos a cada nueva situación. Un ejemplo de este tipo de archivos sería: el archivo de existencias de almacén, que para cada entrada o salida de almacén deberá ser modificado.
- **Archivos históricos.** Se obtienen de los anteriores cuando quedan fuera de uso. Aunque la información que contienen no es de actualidad, reflejan la evolución de la misma. Suelen ser de gran tamaño, y no es frecuente su consulta al azar, su uso habitual es para elaborar estudios estadísticos. Un archivo histórico puede ser el que contiene la información de libros adquiridos por una biblioteca en la década de los ochenta. Otro podría ser, la facturación de una empresa en el año 1.998.
- **Archivos de movimientos.** En ellos se almacena la información que se utilizará para actualizar los archivos maestros. Sus registros, denominados movimientos o transacciones, suelen especificar tres tipos de acciones sobre el maestro: altas, bajas y modificaciones. Una vez realizado el proceso de actualización de un archivo maestro por medio de un archivo de movimientos, éste pierde su validez y podemos deshacernos de él. Su destrucción accidental no es muy grave ya que se puede reconstruir a partir de los documentos que provocaron los últimos movimientos. Estos archivos eran muy utilizados en sistemas *batch* o por lotes. Actualmente, aunque siguen siendo utilizados, se recurre a soluciones más interactivas. Podría utilizarse un archivo de este tipo para actualizar un maestro de almacén con los nuevos artículos, eliminar los artículos descatalogados, modificar el stock de artículos debido a entradas o salidas, o cualquier otro tipo de modificación.
- **Archivos de maniobra o trabajo.** Se utilizan como auxiliares de los anteriores. Tienen una vida limitada, normalmente igual a la duración de la ejecución del programa que los maneja. Son ficheros “puente”, que no serían necesarios si dispusiéramos de una memoria principal infinita. Por ejemplo, si se desea listar un archivo ordenado por alguno de sus campos, se hará por medio de un archivo de maniobra en el que se almacena esta información, allí se ordena por dicho campo, se imprime y posteriormente podríamos eliminarlo.

En la figura siguiente se muestra un ejemplo de actualización de un archivo maestro de almacén mediante un archivo de movimientos:

Archivo maestro de ALMACEN

Cod	Descripción	PVP	Stock
100	Impresora HP 5550	26000	20
104	Teclado ACER	2600	25
115	Pentium IV 1,2	28000	10
120	DIMM 128M	4100	10

Archivo de MOVIMIENTOS

Mov	Cod	Descripción	PVP	Stock
A	095	Fax 56 K	7000	20
B	104			
M	115		27500	
A	118	Minitorre	2500	20
M	120			+5

Nuevo archivo maestro de ALMACEN

Cod	Descripción	PVP	Stock
095	Fax 56 K	7000	20
100	Impresora HP 5550	26000	20
115	Pentium IV 1,2	27500	10
118	Minitorre	2500	20
120	DIMM 128M	4100	15



3.2. Sistemas Gestores de Bases de Datos (SGBD)

5.2.1 Definición de Base de Datos

Una definición breve e informal podría ser la siguiente: “Una colección de datos que están lógicamente relacionados entre sí, que tienen una definición y una descripción comunes y que están estructurados de forma particular”. Una base de datos es también, un *modelo* de una parcela del mundo real, por tanto debe poder servir para una amplia gama de aplicaciones.

5.2.2 Objetivos de los sistemas gestores de bases de datos.

Los objetivos que pretenden cubrir las bases de datos podrían resumirse en los siguientes:

- **Independencia entre datos y aplicaciones:** En una base de datos el programa accede a los datos por medio de un software intermedio que oculta al programa de aplicación la estructura concreta de los datos. Cuando se tenga que cambiar la descripción de un fichero o el formato de sus datos, no habrá que cambiar los programas de aplicación.
- Evitar **redundancias** de datos, es decir, ficheros distintos con información común destinados a diferentes aplicaciones.
- Evitar la **inconsistencia** o **incoherencia** de datos, para ello es fundamental evitar la redundancia de datos en distintos ficheros. La actualización de un dato se hará una única vez en un único fichero físico al que se accede desde cualquier aplicación, y por lo tanto no habrá riesgo de valores inconsistentes para el mismo dato.
- **Centralización del control** de la base de datos mediante un software de control **SGBD** (sistema gestor de la base de datos) y la figura del **administrador de la base de datos**.
- Posibilidad de obtener información de la base de datos sin necesidad de tener que construir programas “a propósito”. Esto puede realizarse gracias a los **lenguajes de consulta** (*Query Language*).
- Preservar la **integridad** de los datos mediante sistemas automáticos de validación, respaldos sistemáticos y procedimientos automáticos de reconstrucción.
- Garantizar la **confidencialidad y seguridad** de los datos contra accesos incorrectos o no autorizados.
- Permitir los **accesos concurrentes** a los datos.

5.2.3 Elementos de un sistema gestor de bases de datos.

1. **Los datos.** Se pueden clasificar en
 - Datos Simples. Ej. : nombre, dirección, teléfono, etc.
 - Datos Compuestos. Ej. : tablas, etc.
2. **El diccionario de datos.** Es una base de datos que contiene datos de interés sobre los datos de la base de datos a la que pertenece. Por ejemplo, puede recoger información sobre la representación física de los datos, asignación a dispositivos, formas de acceso, índices..., además posee el repertorio de todos los campos y variables que existen en la BD, así como descripción, longitud, posibles valores de cada uno de ellos, etc.
3. **El sistema de gestión de la Base de Datos (SGBD).** Conjunto de programas que gestionan la Base de Datos. Las funciones principales son crear, actualizar, consultar, proteger la Base de Datos.

4. **El administrador de la Base de Datos (DBA).** Es una función que pueden llevar a cabo una o varias personas. Entre las responsabilidades del administrador están la de creación, gestión y mantenimiento de la Base de Datos.
5. **Los usuarios.** Todos aquellos que utilizan la Base de Datos. Distinguiendo entre usuarios informáticos, aquellos a los que el administrador le dará todas las herramientas para que puedan realizar su trabajo, y los no informáticos (usuarios finales) a los que va dirigida la Base de Datos.

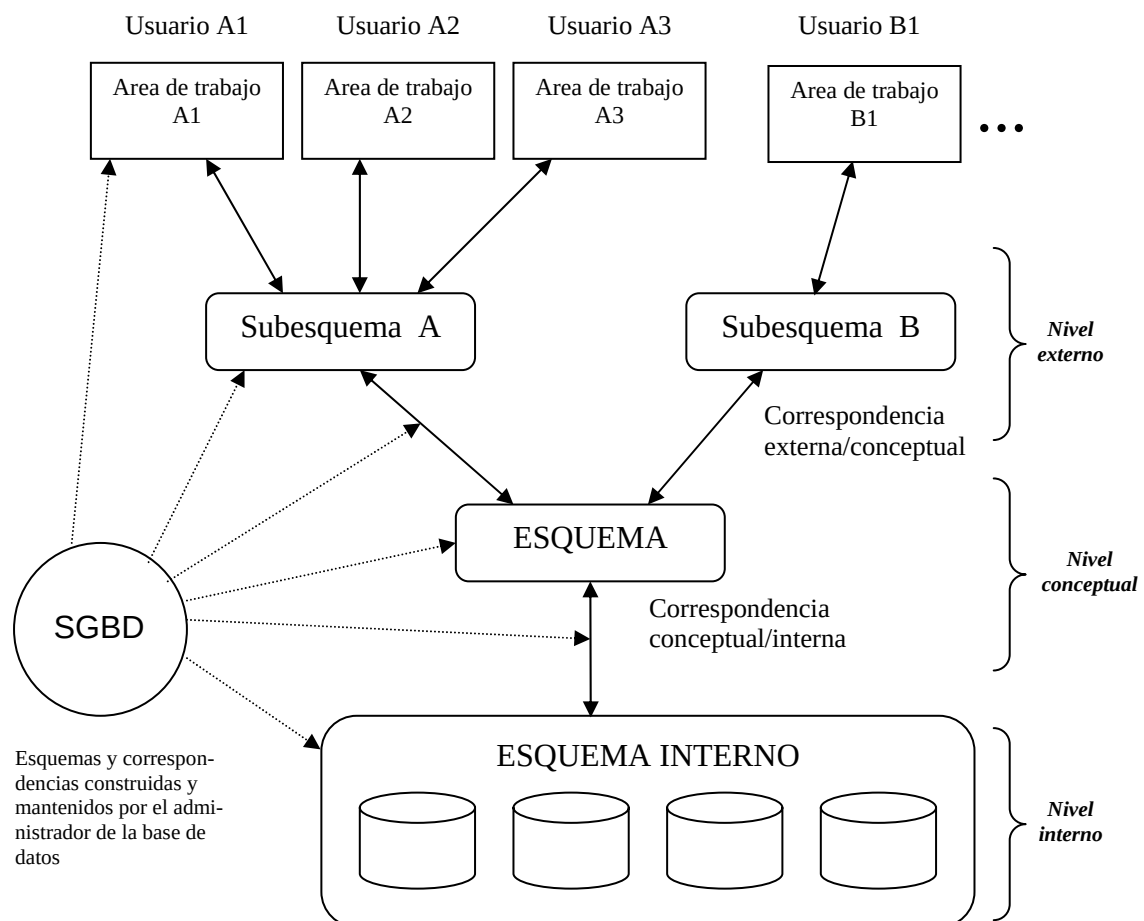
5.2.4 Arquitectura en tres niveles (ANSI – SPARC)

Uno de los objetivos de un sistema de bases de datos es proporcionar a los usuarios una visión abstracta de la información, ocultando ciertos detalles acerca de cómo se almacenan los datos, pero permitiendo una recuperación eficaz de la información.

Existen varios paquetes de sistemas de bases de datos en el mercado con diferente arquitectura. Una de las arquitecturas más estandarizada es la especificada por la normativa ANSI/X3/SPARC (*Standard Planning and Requirements Committee of the American National Standards Institute – Comité de Planificación y Requerimientos de estándares del Instituto Nacional de Estándares de Estados Unidos, división X3*).

Según esta norma la arquitectura de una base de datos debe tener tres niveles: **externo**, **conceptual** e **interno**. En la siguiente figura se muestra la organización de esta arquitectura.

Arquitectura ANSI/X3/SPARC:





Nivel externo.

Es el más cercano a los usuarios. Representa la visión de cada usuario o programador de la base de datos. En el nivel externo se describe de forma individual un subconjunto de datos para un usuario o conjunto de usuarios. Los usuarios pueden trabajar con los archivos externos como si existiesen físicamente, aunque éstos en realidad no existen tal como son vistos por el usuario.

Por cada tipo de usuario es necesario especificar un *esquema externo*, *subesquema* o *vista externa*, que describe un subconjunto de datos de la base de datos. Por lo tanto, existirán tantos subesquemas como tipos de usuarios tenga la base de datos (uno por cada uno de ellos). Habrá usuarios que podrán acceder a más de un esquema externo, y un esquema externo podrá ser compartido por varios usuarios.

Nivel conceptual.

Es una representación de toda la información contenida en la base de datos en una forma más abstracta que la contenida en la estructura interna. Se puede considerar como un modelo teórico de la base de datos sobre el cual estarán asentados los submodelos externos. Se trata de tener una visión global de los datos “como realmente son”, sin estar forzados a definirlos por medio de las restricciones de un lenguaje de programación o un dispositivo hardware concreto.

El administrador es el único que trabaja a nivel conceptual, ya que los usuarios trabajan a nivel externo utilizando subconjuntos de la estructura conceptual.

El objetivo de este nivel es proporcionar una descripción real de la organización independiente de la estructura física de los datos. El nivel conceptual se describe por medio de un *esquema* o *vista conceptual* de la base de datos. El esquema conceptual podría contener:

- Las **entidades** del mundo real (Por ejemplo: empleados, departamentos, componentes, etc. ...)
- Los **atributos** de las distintas entidades (Por ejemplo para la entidad empleado: N°personal, nombre, dirección, DNI, n° hijos, sueldo etc. ...)
- **Relaciones** entre las distintas entidades (Por ejemplo: Un empleado pertenece a un departamento, un departamento contiene varios empleados, etc. ...)
- Podrían incluirse verificaciones de seguridad e **integridad** (Por ejemplo: un empleado no podría pertenecer a un departamento que no existe).

Nivel interno.

Es el más cercano al almacenamiento físico. Se ocupa de cómo los datos se almacenan en el soporte:

- La estructura de almacenamiento físico de los datos.
- La organización de los ficheros.
- Los modos de acceso.
- Los índices y apuntadores.
- El bloqueo de registros.

El nivel interno se describe por medio de un *esquema interno* o *vista interna*. Si el nivel interno y conceptual estuviesen totalmente separados, no se necesitaría de este esquema, ya que podría diseñarlo automáticamente el propio sistema a partir del esquema conceptual. Como no suele ser así, el administrador deberá comunicar al sistema gestor de la base de datos ciertas características respecto a la estructura interna que tiendan a conseguir una mayor eficiencia en el almacenamiento y recuperación de la base de datos.



El administrador es el único que trabaja a nivel interno. Al diseñar el esquema interno se intenta conseguir los siguientes objetivos:

- Disminuir al máximo el tiempo de respuesta del sistema.
- Minimizar el espacio de almacenamiento.
- Evitar en lo posible la redundancia de información.
- Optimización de los recursos de los que se disponen.

El SGBD debe poder garantizar la transferencia de los datos desde el nivel interno al nivel externo, a este proceso se llama transformación de datos o mapeo (*data mapping*). Para ello existen dos niveles de correspondencia:

- **Correspondencia conceptual/interna:** Permite el paso de la vista conceptual a la vista interna, y viceversa. Especifica cómo se representan los registros y campos conceptuales en el nivel interno. Si se modifica la estructura interna de la base de datos, la correspondencia conceptual/interna deberá modificarse, para que no varíe el esquema conceptual. De este modo se conserva la independencia de los datos.
- **Correspondencia externa/conceptual:** Permite el paso de una vista externa específica a la vista conceptual, y viceversa.

Los subesquemas, el esquema conceptual, el esquema interno y las correspondencias conceptual/interna y externa/conceptual, las describe el **administrador de la base de datos** y quedan almacenados dentro del **diccionario** de la base de datos para futuras consultas del SGBD. Cuando un usuario desea acceder a la base de datos, el SGBD examina el diccionario de datos para comprobar si la solicitud puede ser realizada y el modo de realizar las transformaciones pertinentes de los datos.

En la página siguiente vemos un ejemplo de una pequeña base de datos. Se puede observar que se han obtenido dos subesquemas (esquemas externos) a partir del único esquema conceptual formado por las entidades: VENDEDORES, VENTAS y ARTICULOS. La estructura de cada subesquema obedece a las necesidades concretas del tipo de usuario que lo va a utilizar. Por ejemplo:

- a) Uno de los subesquemas será utilizado por un programa que totalizará ventas por departamento, por lo tanto la visión que le interesa tener de la base de datos es una relación de ventas ordenadas por departamentos.
- b) El otro subesquema lo utilizará un programa que listará una relación de empleados por departamento, por lo que sólo necesita conocer los nombres de todos los empleados y el departamento al que pertenecen.



SUBESQUEMAS

Esquema externo o **subesquema** para el programa “Ventas por departamento”, formado por el siguiente *registro externo*:

VENTAS_POR_DEPARTAMENTO

DEPARTAMENTO	CONCEPTO	IMPORTE
--------------	----------	---------

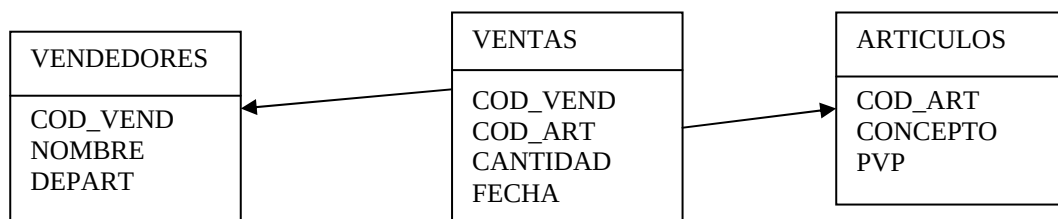
Esquema externo o **subesquema** para el programa “Relación de empleados por departamento”, formado por el siguiente *registro externo*:

VENDEDORES

DEPARTAMENTO	NOMBRE
--------------	--------

ESQUEMA CONCEPTUAL

Esquema conceptual de toda la base de datos, formado por los siguientes *registros conceptuales*:



ESQUEMA INTERNO

Esquema interno de toda la base de datos:

INDICE_VENDEDORES VENDEDORES

CLAVE	PUNTERO	COD_VEND texto(3)	NOMBRE texto(30)	DEPART texto(4)	...
-------	---------	-------------------	------------------	-----------------	-----

INDICE_VENTAS VENTAS

CLAVE	PUNTERO	COD_VEND texto(3)	COD_ART texto(4)	CANTIDAD Entero	FECHA date
-------	---------	-------------------	------------------	-----------------	------------

INDICE_ARTICULOS ARTICULOS

CLAVE	PUNTERO	COD_ART texto(4)	CONCEPTO texto(40)	PVP float	...
-------	---------	------------------	--------------------	-----------	-----

Además se incluiría el tipo de organización, la secuencia física de los registros, modos de acceso, etc.



En la siguiente figura podemos observar algunas ocurrencias de los registros de externos y conceptuales.

VISTAS EXTERNAS

VENTAS_POR_DEPARTAMENTO

DEPARTAMENTO	CONCEPTO	IMPORTE
Automóvil	Alarma Antirrobo	300,00
Automóvil	Faros antiniebla	200,00
Hogar	Juego toallas	80,00
Hogar	Sábanas estampadas	130,00

EMPLEADOS_POR_DEPARTAMENTO

DEPARTAMENTO	NOMBRE
Automóvil	Alfonso Martín García
Hogar	Víctor Fernández Ramírez
Hogar	Pedro Pérez Sánchez

VISTA CONCEPTUAL

VENEDORES

COD_VEND	NOMBRE	DEPARTAMENTO	...
001	Alfonso Martín García	Automóvil	
002	Víctor Fernández Ramírez	Hogar	
003	Pedro Pérez Sánchez	Hogar	

VENTAS

COD_VEND	COD_ART	CANTIDAD	FECHA	...
002	H22	1	03-09-02	
001	A11	1	05-09-02	
003	H15	1	15-09-02	
001	A22	1	22-09-02	

ARTICULOS

COD_ART	CONCEPTO	PVP	...
A11	Alarma Antirrobo	300,00	
A22	Faros antiniebla	200,00	
H15	Juego toallas	80,00	
H22	Sábanas estampadas	130,00	



5.2.5 El diccionario de datos

El diccionario de datos (DD) de una base de datos contiene datos sobre los datos. Constituye lo que se llama una **meta-base**: una base de datos que describe a otra base de datos. El diccionario de datos suele contener la siguiente información:

- Los esquemas externos, conceptual e interno.
- Las descripciones de los registros, campos y relaciones a cada nivel.
- Los procedimientos de autorización de accesos y validación de datos.
- Referencias cruzadas del tipo:
 - Un programa qué datos utiliza
 - Un dato qué programas lo utilizan/modifican.
- Las correspondencias externa/conceptual y conceptual/interna.

El DD es utilizado por el SGBD, y es una valiosa herramienta tanto para el Administrador de la base de datos (le facilita el mantenimiento centralizado de la base de datos), como para usuarios, programadores y analistas de sistemas, para conocer con detalle la base de datos.

5.2.6 SGBD (Sistema Gestor de Bases de Datos)

El SGBD es un conjunto coordinado de programas, procedimientos y lenguajes, que suministra, tanto a los usuarios informáticos, como no informáticos y al administrador, los medios necesarios para describir, recuperar y manipular los datos integrados en la base de datos, asegurando su confidencialidad y seguridad.

Principales funciones del SGBD:

- Facilitar al administrador la descripción de los elementos de datos que integran la base de datos, su estructura y las relaciones que existen entre ellos, así como los controles a efectuar antes de realizar el acceso a la base de datos. Esta función es llevada a cabo por medio de Lenguajes de Definición de Datos (LDD).
- Facilitar la manipulación de los datos, permitiendo a los usuarios de la base de datos añadir, buscar, suprimir o modificar los datos de la misma. Se lleva a cabo por medio de Lenguajes de Manipulación de Datos (LMD).
- Garantizar la seguridad y confidencialidad de la base de datos. Por ejemplo: controlando el acceso de los usuarios a la base de datos mediante palabras de paso y limitando el acceso a los datos mediante un sistema de derechos o privilegios.
- Garantizar la fiabilidad e integridad de los datos. Por ejemplo: No permitir la introducción de fechas erróneas, valores no permitidos, valores incoherentes, relaciones inconsistentes, etc.
- Garantizar la compartibilidad de los datos. Permitir que varios usuarios puedan utilizar los mismos datos sin que se produzcan problemas en los accesos simultáneos.
- Facilitar estadísticas sobre la marcha de la explotación de la base de datos, donde se pueda observar su rendimiento.

5.2.7 Clasificación de los SGBD

Según la estructura lógica, a nivel conceptual o a nivel externo, los SGBD se clasifican en los tres tipos siguientes: jerárquico, red y relacional.

Cada SGBD está diseñado para manejar un tipo determinado de estructura lógica (redes, árboles, tablas). Los programas que se ejecutan bajo un SGBD no se pueden procesar en otro SGBD.

Los SGBD más conocidos de cada tipo, son:

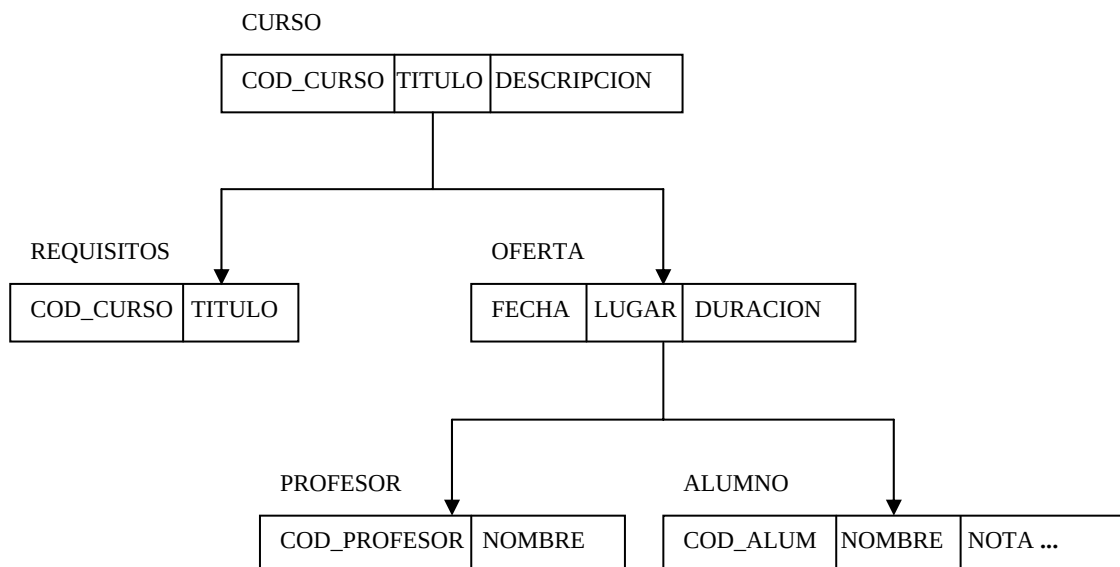
- **Enfoque jerárquico:** El IMS de IBM y el SYSTEM 2000 de Intel.
- **Enfoque de red:** Los ejemplos más importantes los proporcionan las especificaciones del grupo de trabajo de base de datos (DBTG Data Base Task Group) de CODASYL (Conference on Data System Languages), entre los que se encuentran el DMS 1100 de UNIVAC, el TOTAL de Cincom, el EDMS de Xerox, el PHOLAS de Philips, el DBOMP de IBM, y el IDS de Honeywell. Un sistema no-CODASYL es el IMAGE de Hewlett-Packard.
- **Enfoque relacional:** Los más representativos son el SYSTEM R y QBE de IBM, MAGNUM de Tymshare, INGRES de la Universidad de California de Berkeley, ORACLE, INFORMIX, SQL Server de Microsoft.

5.2.7.1 El enfoque jerárquico

Un SGBD de enfoque jerárquico utiliza **árboles** para la representación lógica de los datos. A los archivos que entre sus registros guardan una relación tipo árbol se les llama **archivos jerárquicos**.

La siguiente figura muestra una estructura en árbol que responde a las necesidades de un departamento de formación que gestiona los cursos de formación de sus empleados. Suponemos que cada curso puede celebrarse en distintos lugares y puede ser impartido por varios profesores. Además cada curso exige una serie de requisitos. Los registros utilizados son: CURSO, REQUISITOS, OFERTA, PROFESOR y ALUMNO. El registro CURSO contiene los campos COD_CURSO, TITULO, DESCRIPCION; el registro REQUISITOS contiene el COD_CURSO y TITULO de los cursos necesarios para poder realizar un curso concreto; el registro OFERTA contiene la FECHA, LUGAR, y DURACION de los distintos puntos en los que se va a impartir el curso; el registro PROFESOR contiene el CODIGO y el NOMBRE de cada uno de los profesores que imparten una de las ofertas de un curso, y por último, el registro ALUMNO contiene los campos COD_ALUM, NOMBRE, etc...

Archivo jerárquico:



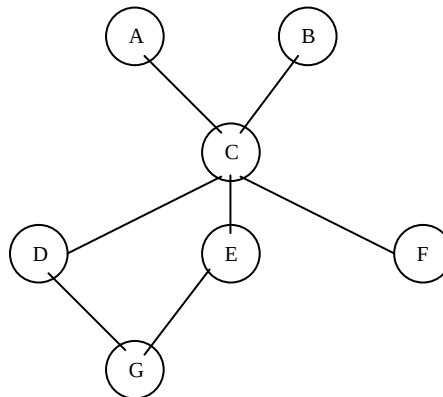
Terminología para describir estructuras en árbol:

- **Raíz:** Es el nodo más alto de la jerarquía (CURSO).
- **Padre:** Es el nodo al que está vinculado otro u otros de nivel inferior. El padre de OFERTA es CURSO, el padre de PROFESOR y ALUMNO es OFERTA.
- **Hermanos:** Son los nodos que tienen el mismo padre; por ejemplo, PROFESOR y ALUMNO.
- **Hijos:** Son los nodos vinculados con otro del nivel superior. Los hijos de OFERTA son PROFESOR y ALUMNO.
- **Hojas:** Reciben este nombre los nodos que no tienen hijos. Son nodos hojas REQUISITOS, PROFESOR y ALUMNO.
- **Camino:** Son las líneas que unen nodos entre sí. Un camino que termina en una hoja se denomina rama.

Todo nodo tiene un único padre, excepto la raíz. Sin embargo, un nodo puede tener muchos hijos.

5.2.7.2 El enfoque en red

Una estructura de datos en red, también llamada estructura *plex*, se caracteriza por que cada nodo hijo puede tener más de un padre, a diferencia de la estructura en árbol, en la que un hijo sólo podía tener un padre.



El nodo C tiene dos padres, A y B; lo mismo sucede con el nodo G, cuyos padres son D y E.

Terminología para describir estructuras en red.

El modelo de red fue propuesto por el comité CODASYL, que es una organización constituida por representantes voluntarios de fabricantes y usuarios de computadoras en la industria y en los Gobiernos de Canadá y Estados Unidos.

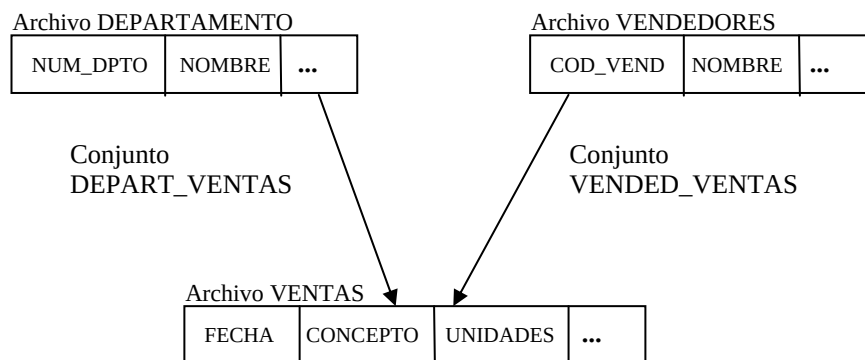
CODASYL definió el **conjunto** (*set*) como la base para describir la relación padre-hijo. Un conjunto está constituido por dos tipos de registro que entre sí tiene una relación uno a muchos.

Los conjuntos tienen las siguientes características:

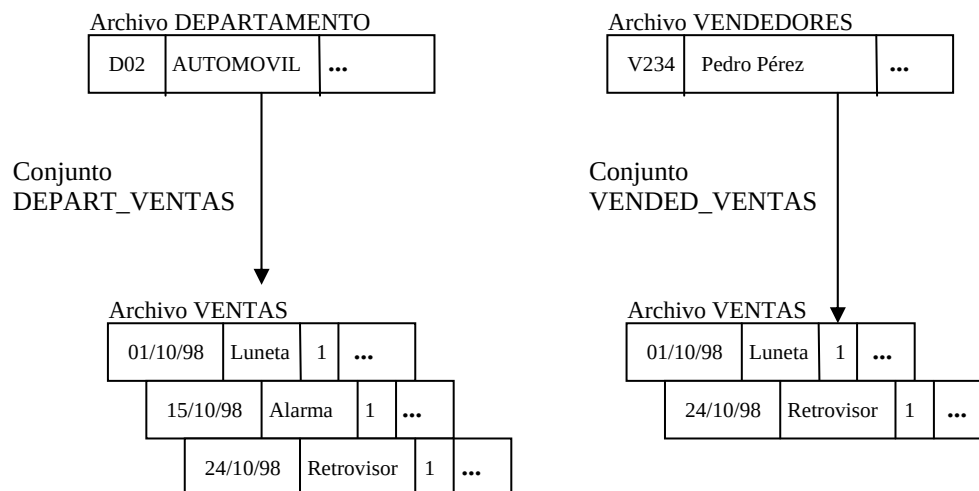
- El registro padre se denomina **propietario** (*owner*) del conjunto, mientras que el registro hijo se denomina **miembro** (*member*).
- Una "ocurrencia de conjunto" es una colección de registros, uno de ellos es el propietario y los otros los miembros.

- El tipo de registro propietario de un tipo de conjunto debe ser distinto de los tipos de los registros miembros. No se admiten *lazos*.
- Sólo se permite que un registro miembro aparezca una vez en las ocurrencias conjuntos del mismo tipo.
- Un registro miembro puede asociarse con más de un propietario, es decir, puede pertenecer al mismo tiempo a dos o más tipos de conjuntos distintos.

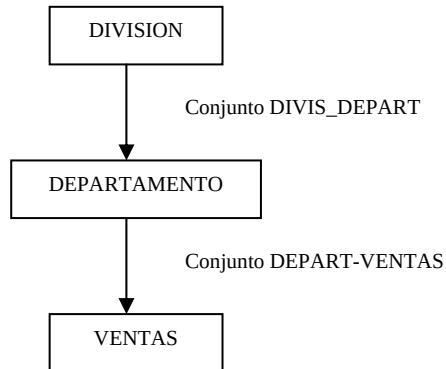
La figura siguiente muestra una estructura tipo red donde el registro VENTAS es miembro de dos conjuntos: DEPART_VENTAS y VENDED_VENTAS.



Un ejemplo para dicha situación aparece en la figura siguiente:



Se pueden definir niveles múltiples de jerarquías donde un tipo de registro puede ser miembro en un conjunto y al mismo tiempo propietario en otro conjunto diferente. La figura siguiente muestra un nivel de jerarquía múltiple donde un registro DEPARTAMENTO se puede declarar miembro del conjunto DIVIS_DEPART y propietario del conjunto DEPART-VENTAS, para una base de datos que contenga información sobre las divisiones, departamento y ventas de una empresa.



5.3. Modelo Relacional de una Base de Datos

5.3.1 Introducción

El modelo de datos relacional es posterior a los modelos jerárquicos y de red. Nació como consecuencia de los trabajos publicados en 1969-70 por E. F. **Codd**, aunque los primeros SGBD relacionales no aparecen en el mercado hasta principios de los años ochenta.

El modelo relacional se ocupa de tres aspectos de los datos: su **estructura**, su **integridad** y su **manipulación**.

La estructura lógica y el modo de realizar las operaciones de Entrada / Salida en el enfoque relacional son distintos respecto a los enfoques jerárquico y de red. El modelo relacional representa la base de datos por medio de tablas llamadas **relaciones**. Algunas de las diferencias con las bases de datos jerárquicas o en red son:

- A la hora de operar con los datos, en lugar de hacerlo sobre registros, actúa sobre la relación (tabla).
- Las bases de datos relacionales no permiten ni grupos repetitivos ni apuntadores en los niveles externo y conceptual, en el nivel interno el sistema tiene libertad para usar cualquier estructura que desee.

5.3.2 Estructura de datos relacional

De manera informal, una base de datos relacional es aquella cuyos usuarios la perciben como un conjunto de tablas. Por ejemplo:

NIF	NOMBRE	LOCALIDAD
1234	Manuel	Melilla
2525	Alfonso	Sevilla
5454	Felipe	Madrid
6776	Pablo	Huelva

a) Tabla de PROVEEDORES

CODIGO	CONCEPTO
10	Monitor
11	Impresora
22	Scanner

b) Tabla de ARTICULOS

COD-ART	NIF-PRO	PRECIO
10	1234	150,25
11	5454	168,28
22	5454	119,00
11	6776	160,20
22	6776	120,15

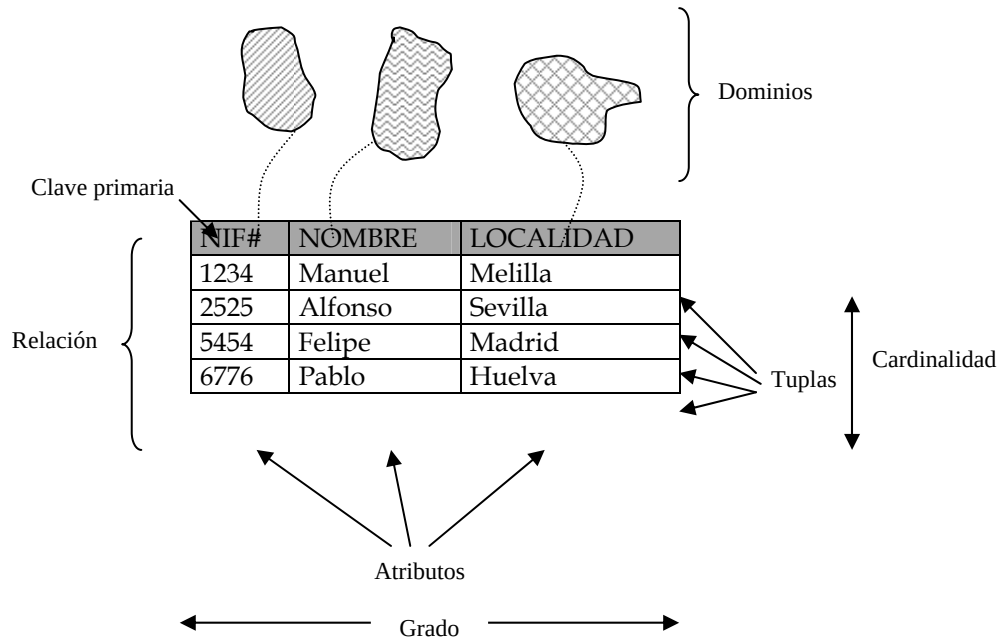
c) Tabla de PRECIOS

El modelo relacional dispone de una terminología propia que pasaremos a describir. Si suponemos que una relación puede representarse mediante una tabla (de hecho es usual y aceptable utilizar el término tabla para referirse a una relación) podríamos decir que:

- Una **tupla** corresponde a una fila de la tabla. Representa cada una de las ocurrencias de la relación (*equivale a lo que conocemos como ocurrencia de un registro, en ficheros clásicos*). El número de tuplas se denomina **cardinalidad**, la cardinalidad varía con el tiempo.
- Un **atributo** corresponde a una columna de la tabla (*equivale a un campo de un registro*). El número de atributos se llama **grado**. El grado no varía con el tiempo, si añadimos un atributo a una relación, podemos considerar que se trata de otra relación nueva. Una relación de grado uno se llama unaria; de grado dos, binaria; de grado tres, ternaria;..., y una relación de grado n, n-aria.
- Un **dominio** es una colección de valores, de los cuales uno o más atributos obtienen sus valores reales. Por ejemplo, el dominio de código de artículo es el conjunto de todos los códigos de artículos posibles (*equivaldría a un conjunto de valores de un tipo dado*).
- **Clave o llave candidata** es un atributo K (o conjunto de atributos) de una relación R que cumple las siguientes propiedades independientes del tiempo:
 - **Unicidad**: No existen dos tuplas en R con el mismo valor de K.
 - **Minimalidad**: Si K es compuesto, no será posible eliminar ningún componente de K sin destruir la propiedad de unicidad.

Por ejemplo, el atributo compuesto (NIF,LOCALIDAD) no es una clave candidata de la relación PROVEEDORES, ya que podemos eliminar el atributo LOCALIDAD sin destruir la propiedad de unicidad, es decir, siguen sin existir dos tuplas con el mismo valor de NIF.

- **Clave o llave primaria**. Es posible que una relación posea más de una clave candidata, en ese caso, se escoge una de ellas como **clave primaria** y el resto se denominan **claves alternativas**. En la práctica la elección de la clave primaria suele ser sencilla. Toda relación, sin excepción, tiene una clave primaria y suele representarse subrayando y/o añadiendo el carácter # al atributo (o conjunto de atributos) correspondiente. Por ejemplo: Artículos(código#, concepto)
- **Clave ajena o extranjera** es un atributo (o conjunto de atributos) de una relación R1 que es clave primaria de otra relación R2. Se utiliza para referenciar a la tupla de R2 cuya clave primaria coincida con el valor de la clave ajena de R1. Ambas claves deben definirse sobre el mismo dominio.
Por ejemplo, el atributo NIF-PROV de la relación PRECIOS es clave extranjera ya que se utiliza para referenciar a una tupla de PROVEEDORES mediante la clave primaria NIF. R1 y R2 pueden ser la misma relación. Por ejemplo, una relación que represente un árbol genealógico de un pedigrí (Código#, Nombre, FechaNac, ...,CodPadre,CodMadre), donde CodPadre y CodMadre son claves ajenas que referencian tuplas de la misma relación.



Ahora podemos dar una definición más formal de una **relación**: Una relación R sobre un conjunto de dominios $D_1, D_2, \dots, D_n$ (no necesariamente distintos) se compone de dos partes: una cabecera o esquema, y un cuerpo.

- La **cabecera** o **esquema** está formado por un conjunto fijo de atributos, o mejor dicho, de pares atributo-dominio $\{(A_1:D_1), (A_2:D_2), \dots, (A_n:D_n)\}$
- El **cuerpo** está formado por un conjunto de tuplas (n -tuplas) que varía con el tiempo. Cada tupla a su vez está formada por un conjunto de pares atributo-valor: $\{(A_1:v_{i1}), (A_2:v_{i2}), \dots, (A_n:v_{in})\}$ ($i=1, 2, \dots, m$ donde m es el número de tuplas del conjunto). Cada v_{ij} es un valor del dominio D_j asociado al atributo A_j .

Además, las relaciones cumplen las siguientes propiedades:

- **No existen tuplas repetidas.** El cuerpo de la relación se ha definido como un conjunto de tuplas, y en matemáticas los conjuntos por definición no incluyen elementos repetidos. Hay que decir que muchos de los SGBD relacionales sí admiten duplicidad de tuplas.
- **Las tuplas no están ordenadas.** El cuerpo de la relación se ha definido como un conjunto de tuplas, y los conjuntos en matemáticas no son ordenados.
- **Los atributos no están ordenados.** La cabecera de la relación se ha definido como un conjunto pares atributo-dominio, y los conjuntos en matemáticas no son ordenados.

Estas propiedades son las que marcan la diferencia entre una tabla y una relación, ya que una tabla presenta las filas y las columnas en un orden, del cual carecen las relaciones. Por otro lado, una tabla podría contener filas repetidas. De todos modos, como dijimos anteriormente, es muy común utilizar el término *tabla* para referirse a una *relación*.

5.3.3 Integridad de los datos

En una base de datos relacional, las relaciones están sujetas a ciertas reglas de integridad:

- **Integridad de entidad:** "Ningún atributo que participe en una clave principal puede tener valores nulos."



Codd justifica esto del siguiente modo: En el modelo relacional, las relaciones representan entidades del mundo real y todas las entidades del mundo real son distinguibles. Las claves primarias llevan a cabo la función de identificación en el modelo relacional, por tanto un identificador nulo sería una contradicción.

- **Integridad referencial:** “No deben existir valores de clave ajena sin concordancia.”

Dicho de otro modo, si una relación tiene una clave ajena que referencia a otra relación (o a ella misma), cada valor de la clave ajena tiene que ser igual a un valor de la clave principal de la relación a la que referencia, o bien, ser completamente nulo.

El diseñador de la base de datos deberá poder especificar qué operaciones han de rechazarse y cuáles han de aceptarse, y en este caso, qué operaciones de compensación hay que realizar para mantener la integridad de la base de datos

5.3.4 Manipulación de los datos en un sistema relacional

5.3.4.1 Lenguaje de procedimientos. Álgebra relacional

Los lenguajes de procedimientos para consultar bases de datos relacionales están basados en el álgebra relacional.

El álgebra relacional consta de un conjunto de operadores de alto nivel que operan sobre las relaciones. Los operandos de cada operación pueden ser una o más relaciones y como resultado dan una nueva relación. La relación resultado se puede someter a nuevas operaciones.

Codd definió ocho operadores de este tipo en dos grupos de cuatro:

- Operaciones tradicionales de conjunto: Unión, intersección, diferencia y producto cartesiano.
- Operaciones relacionales especiales: Proyección, restricción (selección), reunión y división.

Todos los ejemplos de operaciones se realizarán sobre las relaciones de la figura siguiente, que representa un archivo de PROVEEDORES en las relaciones PROV1 y PROV2, un archivo de ARTÍCULOS en la relación ART y un archivo de PRECIOS en la relación PREC. Este último contiene, para cada proveedor, la relación de los artículos que suministra junto con su precio.

NIF	NOMBRE	LOCALIDAD
1234	Manuel	Melilla
2525	Alfonso	Sevilla
5454	Felipe	Madrid
6776	Pablo	Cádiz

a) Relación PROV1

NIF	NOMBRE	LOCALIDAD
2121	Antonio	Almería
2525	Alfonso	Sevilla
5454	Felipe	Madrid

b) Relación PROV2

CODIGO	CONCEPTO
10	Monitor
11	Impresora

c) Relación ART

COD-ART	NIF-PRO	PRECIO
10	1234	150,25
11	5454	168,28
22	6776	120,15

d) Relación PREC



3.3.4.1.1 Operaciones tradicionales de conjunto

Las operaciones con conjuntos son operaciones binarias; es decir, a partir de dos relaciones se obtiene una tercera.

- **operación UNION**

La unión de dos relaciones R y S con el mismo esquema o cabecera, es una relación T con el mismo esquema formada por el conjunto de todas las tuplas que pertenecen a R o a S, o a ambas relaciones. Sin tuplas duplicadas.

La unión se denota por:

$$T := \text{UNION}(R,S) \text{ o bien } T := R \text{ UNION } S$$

La figura siguiente muestra la relación unión de PROV1 y PROV2.

NIF	NOMBRE	LOCALIDAD
1234	Manuel	Melilla
2121	Antonio	Almería
2525	Alfonso	Sevilla
5454	Felipe	Madrid
6776	Pablo	Cádiz

PROV1 UNION PROV2

Observa que las tuplas con NIF 2525 y 5454 no aparecen por duplicado.

- **Operación DIFERENCIA**

La diferencia de dos relaciones R y S con el mismo esquema, es una relación T, con el mismo esquema formada por el conjunto de todas las tuplas que pertenecen a R y no a S.

La diferencia se expresa:

$$T := \text{MINUS}(R,S) \text{ o bien } T := R \text{ MINUS } S$$

La figura siguiente muestra la relación diferencia de PROV1 menos PROV2.

NIF	NOMBRE	LOCALIDAD
1234	Manuel	Melilla
6776	Pablo	Cádiz

PROV1 MINUS PROV2

- **Operador INTERSECCION**

La intersección de dos relaciones R y S con el mismo esquema, es una relación T con el mismo esquema que contiene las tuplas que pertenecen a R y a S a la vez.

La notación utilizada es:

$$T := \text{INTERSECT}(R,S) \text{ o bien } T := R \text{ INTERSECT } S$$

La figura siguiente es el resultado de la intersección de las relaciones PROV1 y PROV2.

NIF	NOMBRE	LOCALIDAD
2525	Alfonso	Sevilla
5454	Felipe	Madrid

PROV1 INTERSECT PROV2

La intersección también puede conseguirse a partir de la diferencia mediante la expresión:
 $INTERSECT(R,S) = R \text{ MINUS } (R \text{ MINUS } S)$

- **Operación PRODUCTO CARTESIANO**

El producto cartesiano de dos relaciones R y S con un esquema cualquiera, es una relación T que tiene por atributos la concatenación de los de R y S, cuyas tuplas son todas las concatenaciones de una tupla de R a una tupla de S. Es decir la relación T contiene todas las combinaciones posibles de tuplas entre R y S.

El producto cartesiano se escribe de la forma:

$$T := TIMES (R,S) \text{ o bien } T := R \text{ TIMES } S$$

La figura siguiente representa un producto cartesiano de PROV2 y ART.

NIF	NOMBRE	LOCALIDAD	CODIGO	CONCEPTO
2121	Antonio	Almería	10	Monitor
2121	Antonio	Almería	11	Impresora
2525	Alfonso	Sevilla	10	Monitor
2525	Alfonso	Sevilla	11	Impresora
5454	Felipe	Madrid	10	Monitor
5454	Felipe	Madrid	11	Impresora

PROV2 TIMES ART

1.1.1. Operaciones relacionales especiales

- **Operación PROYECCIÓN**

El operador de proyección (PROJECT) selecciona ciertas columnas de la relación original y elimina las tuplas duplicadas.

La operación PROJECT se denota:

$$PROJECT(R/X, Y, Z) \text{ o bien } P(X,Y,Z) (R) \text{ o, para abreviar, } R[X,Y,Z], \text{ que se lee: Proyección de la relación R sobre los atributos X,Y,Z.}$$

La figura siguiente representa la proyección de la relación PROV1 a partir del atributo LOCALIDAD.

LOCALIDAD
Melilla
Sevilla
Madrid
Cádiz

PROV1[LOCALIDAD]

- **Operación SELECCIÓN**

La selección (SELECT) de la relación R, de acuerdo con la condición Q, es otra relación con el mismo esquema, en la que sus tuplas son las de R, que cumplen la condición Q. Restringe la relación sólo a las tuplas que satisfagan una condición, de hecho, RESTRINGIR fue el nombre original de esta operación, pero actualmente se conoce como SELECCIONAR (SELECT).

La operación SELECT admite las notaciones:

SELECT(R/condición) o bien R WHERE condición

La figura siguiente representa la selección en la relación PROV1 cuando LOCALIDAD = "Melilla".

NIF	NOMBRE	LOCALIDAD
1234	Manuel	Melilla

PROV1 WHERE LOCALIDAD= "Melilla"

- **Operación REUNION**

El operador de reunión, también llamado unión-j o de yunción (**JOIN**), toma dos relaciones R y S que tengan en común uno o más atributos y crea una nueva relación T concatenando las tuplas que tengan el mismo valor en el atributo especificado. Aunque no es frecuente, podemos realizar reuniones en las que no se exija que los valores sean "iguales", pudiendo ser "mayor que", "menor que", etc.

Esta operación se escribe:

JOIN(R,S/condición) o bien

RYS(iCj) se lee: Reunión de R y S, mediante el operador de comparación C sobre los atributos i de R y j de S. C, puede tomar los valores <, >, =, <=, >=, y <>.

La figura siguiente representa la reunión de la relación PROV1 con la relación PREC cuando el NIF de PROV1 coincide con el atributo NIF-PRO de PREC.

NIF	NOMBRE	LOCALIDAD	COD-ART	NIF-PRO	PRECIO
1234	Manuel	Melilla	10	1234	150,25
5454	Felipe	Madrid	11	5454	168,28
6776	Pablo	Cádiz	22	6776	120,15

JOIN(PROV1,PREC/NIF=NIF-PROV)

La unión presenta varios casos particulares:

- **Equirreunión:** Cuando el operador de comparación toma el valor "igual que". Es el caso del ejemplo anterior.
- **Reunión o producto natural:** Es el tipo de reunión más interesante. Equivale a la equirreunión, pero además se suprimen los **atributos** iguales. Se representa por:

R*S o bien R JOIN S



Si las relaciones no tienen ningún atributo en común, entonces la reunión natural coincide con el producto cartesiano:

$$R \text{ TIMES } S = R \text{ JOIN } S$$

La figura siguiente representa la **reunión natural** de la relación PROV1 con la relación PREC cuando el NIF de PROV1 coincide con el atributo NIF-PRO de PREC.

Observa que no aparece duplicado el atributo NIF.

NIF	NOMBRE	LOCALIDAD	COD-ART	PRECIO
1234	Manuel	Melilla	10	150,25
5454	Felipe	Madrid	11	168,28
6776	Pablo	Cádiz	22	120,15

PROV1 JOIN PREC

- **Operador DIVISION**

Sean R y S dos relaciones de m+n y n atributos, respectivamente; es decir, los atributos n son comunes a las dos relaciones. La división de R entre S es una relación con los atributos m que contiene todos los valores de m en R cuyos valores de n incluyen a todos los valores de n en S. Dicho de otro modo; se define la división entre R y S como el conjunto de todas las tuplas de m atributos, tales que al concatenarlas con todas las tuplas de S siempre producen tuplas contenidas en R.

Admite las notaciones:

$$T := \text{DIVISION}(R,S) \text{ o bien } T := R \text{ DIVIDEBY } S$$

Las siguientes figuras representan ejemplos de división de la relación ART-PROV (es una proyección de PREC) entre la relación DVSOR.

COD-ART	NIF-PRO
10	1234
11	5454
10	2525

ART-PROV

COD-ART
10

DVSOR

NIF-PRO
1234
2525

RESULTADO

Las cinco operaciones básicas del álgebra relacional son: **unión**, **diferencia**, **producto cartesiano**, **proyección** y **selección**. Sin embargo, si se utilizan sólo estas operaciones, algunas consultas resultan muy costosas. Por esta razón se definen operadores adicionales (reunión, intersección y división), que no hacen más potente al álgebra relacional pero sí simplifican algunas operaciones de consulta, como por ejemplo la **reunión natural**. Algunos autores proponen nuevos operadores que proporcionan al álgebra la capacidad de cálculo, ya que en la práctica resulta muy interesante. Por ejemplo:

- Un operador que tome una relación y genere otra con atributos cuyo valor se obtenga al operar con atributos de la relación inicial.
- Un operador que tome una relación y genere otra que totalice el valor de ciertos atributos.



5.3.4.2 Lenguaje sin procedimientos. Cálculo relacional

El cálculo relacional se basa en una rama de la lógica matemática llamada cálculo de predicados. El concepto de un **cálculo relacional** (cálculo de predicado aplicado a las bases de datos relacionales) fue propuesto por vez primera por Codd, además propuso un lenguaje basado en ese cálculo llamado “sublenguaje de datos ALPHA”. ALPHA nunca se llevó a la práctica, pero el lenguaje QUEL de INGRES es muy parecido a él. Mediante este lenguaje el usuario expresa lo que desea obtener pero no cómo obtenerlo.

Tanto el álgebra como el cálculo relacional proporcionan una base para el estudio de la parte manipulativa del modelo relacional. La diferencia entre ellas es que el **álgebra** ofrece un conjunto de operaciones explícitas (reunión, unión, proyección, etc.) que puede servir en la práctica para indicar al sistema la forma de construir una relación a partir de otras, y el **cálculo** sólo ofrece una notación para definir la relación deseada en términos de otras relaciones.

Según lo que representen las variables utilizadas, el cálculo relacional se puede clasificar en:

1. Cálculo relacional de tuplas.
2. Cálculo relacional de dominios.

Ambos tipos son similares, y la diferencia fundamental radica en que las variables del primero representan tuplas y las del segundo representan dominios.

5.4. Sistemas Gestores de Bases de Datos Documentales

Sistemas de gestión de bases de datos documentales: son sistemas que incorporan todas las características de los SGBD tradicionales, incluyendo la creación y mantenimiento de bases de datos documentales, usuarios, controles de seguridad, e incluso lenguajes propios de programación. Debe citarse BRS/Search, de BRS Information Technologies (uno de los más completos), Inmagic, CDS-Isis y su interfaz WinIsis, Texto...